Retina Illness Identification

1 Introduction

In the following work, we are asked to build an algorithm capable of analyzing an Optical Coherence Tomography (OCT) and detect one of three pathologies : a (0) choroidal neovascularization, a (1) diabetic macular edema, or a (2) drusen, or identify a (3) healthy retina. In order to achieve this, we are given a dataset of about 97 000 labelled images. Each image has a resolution of 28 by 28 pixels, has a pixel intensity varying between 0 and 1, and is subject to simple linear transformation such as rotation or mirroring. The test set is composed of 1000 of these images with no information regarding their labelling.

To accomplish this task, we first demonstrate that even a roughly tuned simple Convolutional Neural Network (CNN) is sufficient to achieve an accuracy of 56.5% (57.3%) on the public (private) leaderboard, a performance well above that of the logistic regression algorithm. Secondly, with a slightly more sophisticated CNN architecture and some fine-tuning of the hyperparameter (HP), we demonstrate that it is possible to obtain an accuracy of 75.5% (80.1%) on the public (private) leaderboard. A few preprocessing techniques are tried on the second CNN with variable success.

2 Feature Design

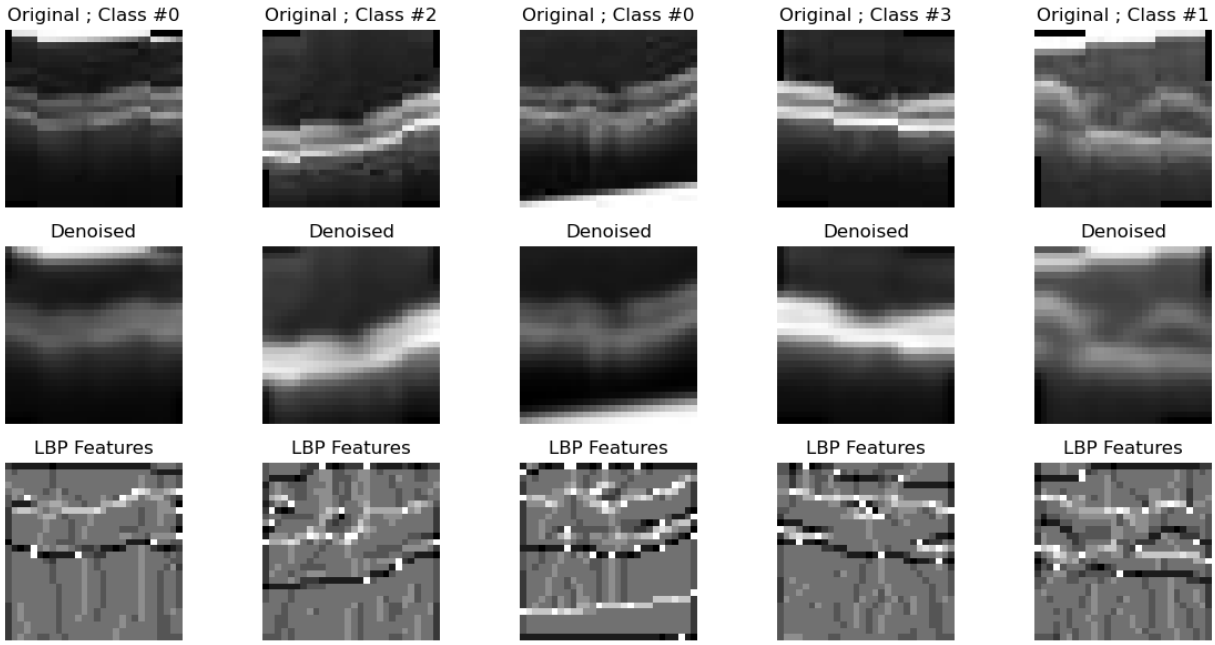


Figure 1: Some images from the dataset. The top row shows the raw signal. The middle row shows them filtered through a Gaussian filter ($\sigma = 1$). The bottom row shows the associated Local Binary Pattern (LBP) of the filtered image.

Three preprocessing algorithms are tested to see if we can improve performance. First, we treat the data with a Gaussian filter (σ=1). We hypothesize that this could mellow out some artifacts and reduce the risk of overfitting.

Second, a Local Binary Pattern (LBP) algorithm is used on the filtered images. We hypothesize that the contrast gained from these images could help identify the contour and important features of the retina [1]. These images are then fed in tandem to the CNN. Figure 1 shows examples of each class that goes through this process. We can see the Gaussian filters helps reduce the background noise and that the LBP algorithm identifies the retina's *ridges* with variable success, but also seem to pick up on some artifacts. When using these pre-processing techniques, an under-sampling algorithm is used to compensate for the imbalance in the class distribution. This algorithm cuts the amount of training examples within a class is to the number of examples of the class with the least representation in our dataset. In our case, this specifically means that the training data is reduced from (0: 26 847 | 1: 8148 | 2: 6207 | 3: 36 779) examples to (0: 6207 | 1: 6207 | 2: 6207 | 3: 6207) examples. Third, a bootstrapping-like algorithm is used reclaim the unused portion of the training data. After the training-validation split, this algorithm picks the least represented class in the data (in our case, class #2 with its 6207 examples), and picks an equal number of randomly chosen points in each other class. This process is repeated a number of time, until the total sampled data in each class corresponds roughly to the number of examples in the most represent class. For example, if we have two classes 1 and 2, where class 1 has 10 examples, and class 2 has 56 examples, we would make 6 ($56/10 \approx 6$) sets of data of 20 data points, with 10 examples from each class.

3 Algorithms

The simple stochastic neural network, written only with NumPy, is made with the following architecture, and takes as input the raw image of the OCT :

- A 2-dimensional convolution followed with a ReLU activation function,
- A max pooling layer with a 2x2 filter and a stride of 2,
- A flattening step,
- A fully connected layer of neurons followed with another ReLU activation function,
- A final fully connected layer into a logits vector.

The second neural network follows a slightly more sophisticated design, and is adjusted to take in any combination of the raw, denoised, and LBP image of the OCT (see figure 1):

- A 2-dimensional convolution followed with a ReLU activation function,
- A max pooling layer with a 2x2 filter and a stride of 2,
- A dropout layer set at 0.25 to clean up the image of any noise and reduce the risk of overfitting,
- A second 2-dimensional convolution followed with a ReLU activation function,
- A max pooling layer with a 2x2 filter and a stride of 2,
- A dropout layer set at 0.25 to clean up the image of any noise and reduce the risk of overfitting,
- A flattening of the data into a 1D vector,
- A fully connected layer of neurons followed with another ReLU activation function,
- A final fully connected layer into a logits vector with a softmax activation function.

4 Methodology

We always start by splitting 20% of the data set into a validation set, and leaving the rest for training. Since we are dealing with an unbalanced class distribution, one of two techniques are used to avoid overfitting on a single class. Either we undersample the data set, either we bootstrap it in order to have multiple smaller but balanced training sets. For setting HPs, we independently analyze the impact of each HP on the performance of the CNN with the validation set. This approach might lead to suboptimal parameters, as co-dependence between these parameters

might lead an uneven and complex optimization landscape, but this is necessary if we want to keep this approach reasonably short.

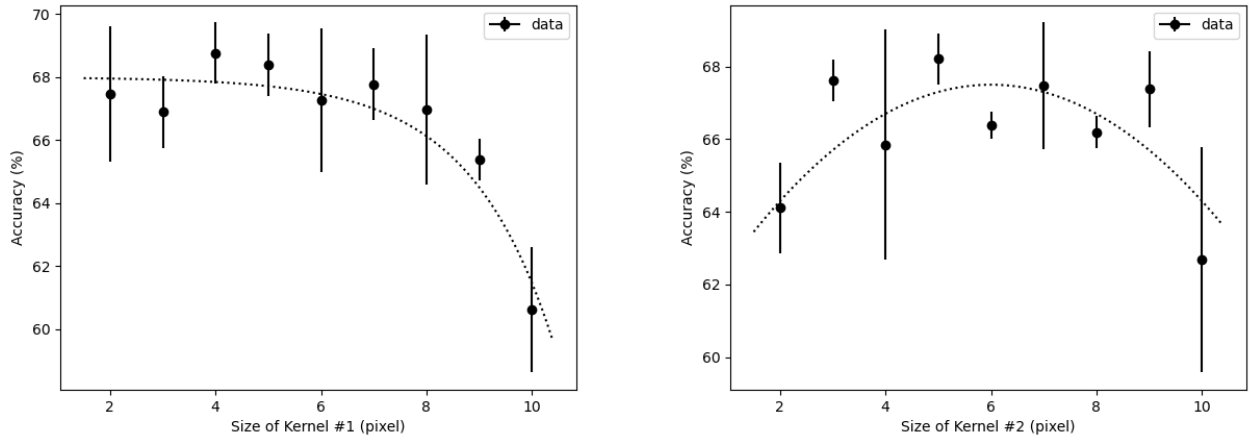
We do this for a large number of kernel size values and a small number of filter values (to draw a comparison between a low- and a high-capacity CNN). We also look at how the CNN's input affects its performance. To do this, we try different combinations of inputs with the raw, filtered, and LBP images. Finally, we implement some bootstrapping to train with a greater fraction of the dataset and we compare different voting techniques.

5 Results

Milestone 1

With its modest single convolutional layer with a kernel width of 3 pixels and single fully connected layer with 64 neurons, this CNN written in NumPy achieves an accuracy of 56.5% (57.3%) on the public (private) leaderboard after 15 epochs of training, considerably outperforming the logistic regression model. Although this model has relatively limited capacity, it does show great potential even with no optimization of its HPs. However, it is greatly inefficient and even with a small number of filters and neurons, a single epoch can take up to a few minutes in calculation time. It is therefore logical to change platform and work with TensorFlow for the rest of this work.

Milestone 2



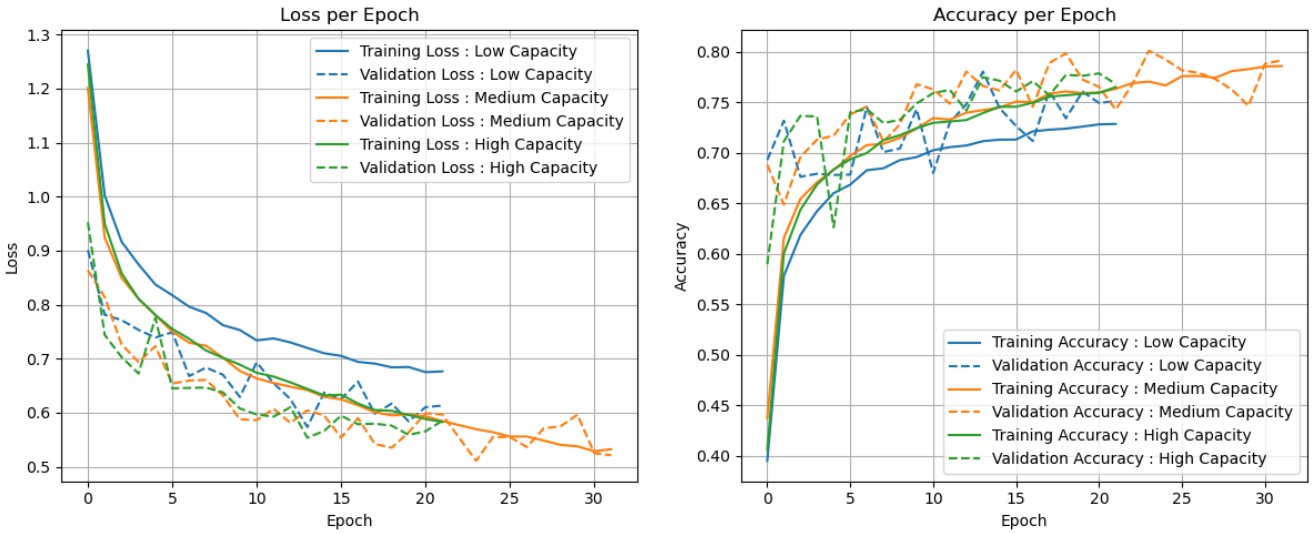
(a) (b)

Figure 2: (a) Validation accuracy for different kernel widths in the first convolutional layer. The second kernel has an arbitrary width of 3 pixels. (b) Validation accuracy for different kernel widths in the second convolutional layer. The first kernel uses a width of 7 pixels. In all cases, we 0-pad the images in order to preserve the shape of the input. The dotted lines are not fitted functions and merely serves as a guide to the eye. The rest of the CNN's architecture goes as follows : 32 filters out of the first convolutional layer, 64 filters out of the second convolutional layer, and 128 neurons in the fully connected layer.

Figure 2 shows the accuracy of the CNN for different kernel size in its two convolutional layers. The accuracy depicted in the graph is obtained with the validation set by averaging the CNN's performance on the last four epochs. Similarly the error bars are obtained by computing the standard deviation on the last four epochs. The algorithm either stops once the CNN has completed either 40 epochs, or hasn't seen an improvement in validation performance in 8 epochs. In sub-figure a), we see a clear decrease in performance when the kernel size surpasses 8 pixels in width. In order to maximize the capacity of our network, and avoid any underfitting, we choose to the

greatest HP value possible without losing too much performance. This sweet spot happens at a kernel width of 7 pixels. In sub-figure b), we repeat the same process, but this time with the trend changes up a bit. Instead of seeing a plateau on one side, we see decreasing performances on either end of the graphic. Since the best performance sits at kernel width of 5 pixels, this value is chosen for the following tests.

With our optimized kernel widths, we move on to test the capacity of our model. In figure 3, three models with varying capacity (low, medium and high) have their loss function and accuracy shown as a function of epochs, and computed over both the training and validation dataset. We start by noting by the end of the learning process, the model with a moderate capacity boasts the best accuracy, both on the training and validation set. We also note that this model takes a few more epochs before early stopping triggers. A final thing worth noting is that the training loss of the medium and high capacity model almost perfectly overlap on each other, seemingly indicating that any increase in *brute* capacity doesn't substantially increase performance, and only increasing calculation time.

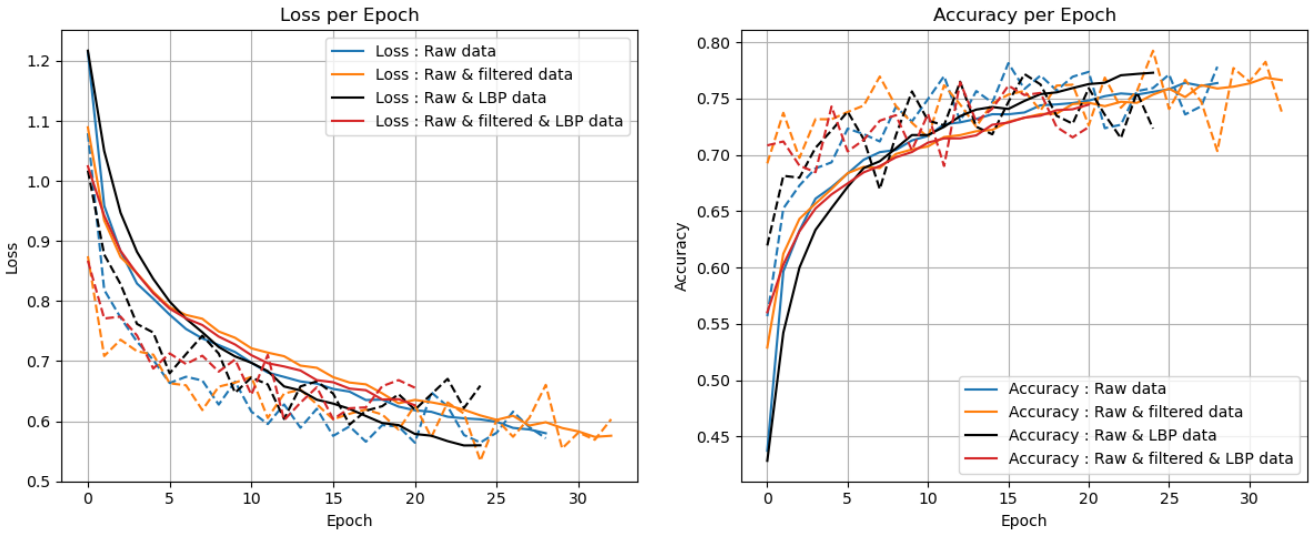


(a) (b)

Figure 3: (a) Training and validation loss comparison between a high capacity algorithm and a low capacity algorithm. (b) Training and validation accuracy comparison between a high capacity algorithm and a low capacity algorithm. The three CNN have respectively (low/medium/high) 16/32/64 and 32/64/128 filters coming out of the two convolutional layers and 64/128/256 neurons in the fully connected layer. The rest of the architecture is identical to figure 2.

Moving on, we now consider how pre-processing techniques on the raw data could increase performance. Figure 4 shows the training process our algorithm with its optimized HPs, as well as the training process of 3 other algorithms, all identical except for their input data. Each new algorithm always takes in the raw image we are trying to classify, but can also include its filtered self, the local binary pattern image obtained from the associated algorithm, or both. The final performance on the validation set of each algorithm is listed in table 1.

The first conclusion that jumps out from these results is that within statistical uncertainty, no gains in accuracy are made. This suggests that the raw data already contains all the relevant information. We can push this analysis further and state that the filtered has no effect on the algorithm's processing, as no significant change in performance can be seen, but that the LBP images have a negative effect on the performance (most likely by adding random noise to the image, making subtle patterns less obvious), as can be seen by the $\approx 3\%$ decrease in performance whenever we include them in the input data. Moreover, in the two algorithms where the LBP images are involved, hints of overfitting are seen earlier in the training process. For instance, the validation loss of the black and red curve overtake the training loss at 13th and 11th epoch respectively, as opposed to the blue and orange curves, which see early signs of overfitting at the 21st and 22nd epoch.



(a) (b)

Figure 4: (a) Training and validation loss comparison for four different inputs objects. (b) Training and validation accuracy comparison for four different inputs objects. The full line represents values calculated on the training set whilst the dashed lines are calculated with the validation set.

Input data	Average performance after training (%)
Raw	75.7±1.8
Raw & Filtered	76.5±1.7
Raw & LBP	73.2±1.5
Raw & Filtered & LBP	73.0±1.5

Table 1: Different input data and their performance with the validation dataset. *Raw* data refers to the untreated data, *Filtered* data refers to the raw images than have gone through a Gaussian filter, and *LBP* data refers to the images obtained with the help of the local binary pattern algorithm.

We also try to implement a bootstrapping algorithm to increase the accuracy of our CNN as is described in section 3. Two voting techniques are tried and listed in table 2. We can see that both approaches offer a $\approx 2\%$ increase in performance across both the public and private leaderboard, which seems rather underwhelming (although note that this represents a modest $\approx 10\%$ decrease in miss classifications). The similarity in performance from these approaches is most likely an indication that the output logits closely mimics the *one-versus-all* format, meaning we are mostly guessing the same. This is easily verifiable and doing so with the test set reveals that the two techniques make the same guesses on 94.7% of images. The remaining 5% are most likely misclassified by both techniques therefore leading to the same error rate. All in all, bootstrapping offers a simple way to reduce the variance of our model.

Voting technique	Public performance (%)	Private performance (%)
No bootstrap	73.8	78.1
Simple summation	75.5	80.1
Majority vote	75.0	80.5

Table 2: Different voting techniques and their performance on the public and private leaderboard.

Finally, we can comment on the mistakes our algorithm makes. Looking at the confusion matrix in table 3, we

can a more global picture of what is going on. For instance, true class 2 images are rarely misclassified, but on the other hand, an important portion of true class 1 and true class 3 images are misclassified as class 2 images. This is so pronounced that only 28% of class 2 guesses are correct. Briefly said, with this representation, we can confidently state that any further effort to increase accuracy should target the greatest off-diagonal terms, namely, true class 3 images classified as class 2 images.

Predicted class (↓) | True class (→)	Class 0	Class 1	Class 2	Class 3
Class 0	0.280	0.003	0.003	0.004
Class 1	0.016	0.087	0.003	0.010
Class 2	0.044	0.007	0.058	0.095
Class 3	0.003	0.007	0.015	0.364

Table 3: Normalized confusion matrix from the bootstrapped predictions. The true validation data class distribution is (0: 34% | 1: 11% | 2: 8% | 3: 47%)

6 Discussion

With the evidence laid in the previous section, it is reasonable to think we have reached the limitations of our approach. To push this algorithm further would require the implementation of new techniques, as the HP tuning was quite thorough. Among those, a few ideas stand out as reasonable hypotheses. Firstly, introducing a regularization term in the gradient descent, such as an L1 or L2 regularization, could have helped with performance. We demonstrated in figure 4 that the CNN converged fairly quickly, and that overfitting became an issue by the 20th epoch in the best scenario. A smaller learning rate with a regularization term could have helped learn at a slower pace and found a better local minima. Secondly, some (more) well thought out feature extraction could help with performance, although it remains unclear whether there really was a better way of presenting the data. This is where expertise in the field could be used to build robust priors on some features. These priors could specifically help target the miss classifications identified in table 3. Finally, it could have been interesting to study how the CNN reacted to small random rotations (or mirroring) on the validation dataset, without tampering with the training set. This could have identified a weakness in the training process since this is an operation we know was made on the dataset. Building a tolerance do these actions could reasonably increase performance.

In conclusion, with a proper architecture with sufficient capacity, some HP tuning, and some bootstrapping, we were able to successfully identify 75.5% (80.1%) on the public (private) leaderboard. As is attested by the confusion matrix on the best performing model, identifying the cause for the overestimation of true class 3 images as class 2 images could push our performance a few percentage points higher and realistically offers the best chance at a considerable improvement in our algorithm. Some other avenues of improvements for this were underline, but no obvious path is clear as of now.

References

[1] Z. Ma, Y. Ding, B. Li, and X. Yuan, "Deep CNNs with Robust LBP Guiding Pooling for Face Recognition", en, Sensors **18**, Number: 11 Publisher: Multidisciplinary Digital Publishing Institute, 3876 (2018) 10.3390/s18113876.